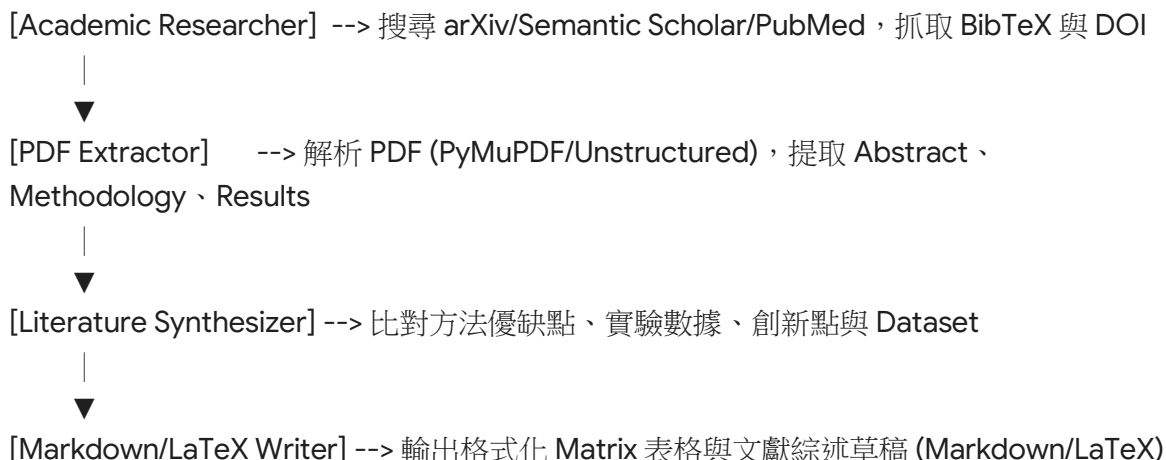


在攻讀博士（PhD）期間，文獻檢索與綜述（Literature Review）往往需要極高的高度與準確性。使用 **CrewAI** 構建多 **Agent** 協作系統來處理 SCI 論文，能將原本繁重的「搜尋、下載、剖析、比較與整理」工作大幅自動化。

1. 架構設計與 Agent 分工 (Architecture)

設計 4 個專門的 **Agent**，採取 **Sequential**（順序）或 **Hierarchical**（層級）流程 協作：



1. Academic Researcher Agent（論文檢索員）

- **職責：**根據指定關鍵字或研究題目，調用 API 檢索近幾年 SCI/頂級會議文獻。
- **工具：**Semantic Scholar API、ArXiv API、Serper/Google Scholar API。

2. PDF Extractor Agent（內文解析員）

- **職責：**讀取全文/PDF，精準擷取「研究背景、核心方法 (Methodology)、實驗數據 (Datasets/Metrics)、局限性 (Limitations)」。
- **工具：**PyMuPDF、Unstructured 或 LlamaIndex / LangChain PDFLoader。

3. Literature Synthesizer Agent（學術分析與綜合員）

- **職責：**交叉比對多篇論文，找出研究缺口（Research Gap）、方法論演進脈絡與異同點。

4. LaTeX / Markdown Writer Agent（報告生成員）

- **職責：**將分析結果彙整為 Markdown 筆記、BibTeX 引用檔，並自動生成 Summary Matrix（文獻比較矩陣表）。

2. 關鍵工具集 (Tools Setup)

博士研究講求 **學術準確度**，切忌讓 LLM 產生幻覺（Hallucination）。因此 Agent 必須搭配強力的外部工具：

- **檢索 API：**

- **Semantic Scholar API：**免費且包含豐富的 Citation、Abstract 及 S2ORC 全文數據。

- ArXiv API：資工/理工領域必備。
- PubMed API：生醫領域適用。
- **PDF 解析庫：**
 - 使用 PyMuPDF (fitz) 或 pdfplumber 抽取特定章節（如 Section 3 Methodology）。
- **結構化輸出 (Pydantic / JSON)：**
 - 強制 Agent 按照固定 Schema（如：Title, Authors, Year, Method, Dataset, Performance, Limitations）輸出，方便後續直接入庫或轉存 Excel/Notion。

3. CrewAI 核心程式碼示範

以下為使用 Python 建立 CrewAI 論文自動化流程的基本模板：

```
import os
from crewai import Agent, Task, Crew, Process
from crewai.tools import tool

# 1. 定義自訂工具：搜尋 Semantic Scholar / arXiv API
@tool("Search Academic Papers")
def search_academic_papers(query: str) -> str:
    """根據關鍵字搜尋相關學術論文並回傳 DOI, Title, Abstract, Year"""
    # 此處接入 Semantic Scholar API 或 ArXiv API
    # 範例回傳結構化文字
    return f"Found papers for query: {query} ... [API Response]"

@tool("Extract PDF Content")
def extract_pdf_content(pdf_url_or_path: str) -> str:
    """提取論文 PDF 檔案的詳細內容 (特別是 Methodology 與 Results) """
    # 此處整合 PyMuPDF 或 pdfplumber 進行文字解析
    return f"Extracted text from {pdf_url_or_path}..."

# 2. 定義 Agents
researcher = Agent(
    role="Senior Academic Researcher",
    goal="搜尋最新且高引用的 SCI 論文，挑選符合 {research_topic} 的核心文章",
    backstory="你是一位資深研究人員，擅長評估論文的學術價值與品質。",
    tools=[search_academic_papers],
    verbose=True
)

pdf_analyzer = Agent(
    role="Paper Content Extractor",
    goal="深入剖析論文 PDF，精準提煉研究方法、數據集、實驗結果與局限性",
    backstory="你是文獻閱讀專家，能迅速從數頁論文中抓出核心貢獻與關鍵細節。",
    tools=[extract_pdf_content],
    verbose=True
)
```

```

)

synthesizer = Agent(
    role="Literature Review Synthesizer",
    goal="整理所有論文，生成 Literature Review Matrix 表格並分析 Research Gap",
    backstory="你擅長學術寫作，能夠將多篇獨立論文整理成邏輯嚴密的綜述矩陣。",
    verbose=True
)

# 3. 定義 Tasks
task_search = Task(
    description="搜尋關於 {research_topic} 的近 5 年 SCI 論文，整理出 5-10 篇最相關的清單。",
    expected_output="包含 Title, Authors, Year, DOI, Abstract 的列表。",
    agent=researcher
)

task_extract = Task(
    description="讀取上一階段挑選的論文，提取其核心方法 (Methodology)、實驗數據與 Limitation。",
    expected_output="每篇論文的結構化摘要。",
    agent=pdf_analyzer
)

task_synthesize = Task(
    description="將所有提取出的資訊彙整為 Markdown 格式的 Summary Table，並寫出一份 500 字的 Research Gap 報告。",
    expected_output="一份包含對比表格與學術缺口分析的 Markdown 文檔。",
    agent=synthesizer
)

# 4. 組成 Crew 並執行
paper_crew = Crew(
    agents=[researcher, pdf_analyzer, synthesizer],
    tasks=[task_search, task_extract, task_synthesize],
    process=Process.sequential
)

result = paper_crew.kickoff(inputs={"research_topic": "Graph Neural Networks for Semiconductor Layout Analysis"})
print(result)

```

4. PhD 實用建議與優化思維

1. 整合文獻管理工具 (Zotero / EndNote)：

- 可以編寫自訂 Tool，讓 Agent 讀取完論文後直接調用 pyzotero 自動寫入你的 Zotero 資料庫，並加上特定 Tag（例如 #CrewAI-Processed）。

2. 結構化矩陣 (Literature Matrix) :

- 讓撰寫 Agent 將輸出格式設定為 Markdown Table 或 CSV，欄位建議設計為：
| Paper Title | Year | Core Method | Dataset & Metrics | Main Findings | Limitations/Gap |

3. 人類在環 (Human-in-the-Loop) :

- 在篩選論文階段，可開啟 CrewAI 的 human_input=True 功能。Agent 找到 20 篇論文後暫停，由你確認「這 5 篇符合我的博士方向」後，再讓 Agent 繼續進行深度的 PDF 解析與彙整。

4. 模型選擇 (LLM Models) :

- 建議使用推理能力強且上下文窗口較大的模型（如 GPT-4o, Claude 3.5 Sonnet 或 DeepSeek-R1）作為內文剖析與 Synthesizer 的 LLM Brain。